

LABORATORY ASSIGNMENT

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 1

Student Name: R.PUJITHARAM

Roll No.: A24126552109

1. TITLE

Implementation of Object-Oriented Inheritance Types in JavaScript

2. AIM

To implement and demonstrate different types of inheritance in JavaScript, including single, multilevel, hierarchical, hybrid, and multiple inheritance using classes and objects.

3. OBJECTIVE

The objectives of this assignment are:

- To understand the concept of inheritance in JavaScript.
 - To implement single inheritance.
 - To implement multilevel inheritance.
 - To implement hierarchical inheritance.
 - To understand hybrid inheritance using mixins.
 - To simulate multiple inheritance using `Object.assign()`.
 - To reuse properties and methods from parent classes.
 - To understand the use of `extends` and `super()`.
-

4. THEORY

Inheritance is an object-oriented programming concept that allows a class or object to acquire properties and methods from another class or object.

In JavaScript, the `extends` keyword is used to create a child class from a parent class. The `super()` keyword is used to call the constructor of the parent class.

The program demonstrates:

- **Single Inheritance:** One child class inherits from one parent class.
- **Multilevel Inheritance:** A class inherits from another derived class in multiple levels.

- Hierarchical Inheritance: Multiple child classes inherit from the same parent class.
 - Hybrid Inheritance: A combination of inheritance with a mixin.
 - Multiple Inheritance: JavaScript does not directly support multiple class inheritance, so `Object.assign()` is used to copy methods from multiple objects.
-

5. ALGORITHM / PROCEDURE

- Create a JavaScript program.
 - Define a parent class and child class for single inheritance.
 - Use `extends` and `super()` to inherit properties and methods.
 - Create three classes for multilevel inheritance.
 - Create multiple child classes from one parent class for hierarchical inheritance.
 - Create a mixin and assign its method to a class prototype for hybrid inheritance.
 - Create two objects containing methods for multiple inheritance.
 - Use `Object.assign()` to add both methods to the child class prototype.
 - Create objects of the required classes.
 - Call the inherited and class-specific methods.
 - Execute the program and verify the output
-

6. PROGRAM

```
console.log("\n===Single Inheritance===");
class Employee {
  constructor(name, salary) {
    this.name = name;
    this.salary = salary;
  }
  display() {
    console.log(`Name: ${this.name}`);
    console.log(`Salary: ${this.salary}`);
  }
}
class Manager extends Employee {
  constructor(name, salary, department) {
    super(name, salary);
    this.department = department;
  }
}
```

```
}  
work() {  
    console.log(`${this.name} manages the ${this.department} department`);  
}  
}  
  
const m = new Manager("Rahul", 50000, "IT");  
m.display();  
m.work();  
  
console.log("\n===Multilevel Inheritance===");  
class Pet {  
    constructor(name) {  
        this.name = name;  
    }  
    eat() {  
        console.log(this.name + " eats");  
    }  
}  
  
class Dog extends Pet {  
    constructor(name, breed) {  
        super(name);  
        this.breed = breed;  
    }  
    bark() {  
        console.log(this.name + " barks");  
    }  
}  
  
class Puppy extends Dog {  
    constructor(name, breed, age) {  
        super(name, breed);  
        this.age = age;  
    }  
}
```

```
}  
  play() {  
    console.log(this.name + " plays");  
  }  
}  
  
const p = new Puppy("Tommy", "Beagle", 1);  
  
p.eat();  
p.bark();  
p.play();  
  
console.log("\n===Hierarchical Inheritance===");  
class Vehicle {  
  constructor(brand) {  
    this.brand = brand;  
  }  
  start() {  
    console.log(this.brand + " vehicle starts");  
  }  
}  
  
class Car extends Vehicle {  
  constructor(brand, model) {  
    super(brand);  
    this.model = model;  
  }  
  drive() {  
    console.log(this.brand + " " + this.model + " is driving");  
  }  
}  
  
class Bike extends Vehicle {  
  constructor(brand, model) {  
    super(brand);
```

```
        this.model = model;
    }
    ride() {
        console.log(this.brand + " " + this.model + " is riding");
    }
}

const car = new Car("Toyota", "Camry");
const bike = new Bike("Honda", "Shine");

car.start();
car.drive();

bike.start();
bike.ride();

console.log("\n===Hybrid Inheritance===");
class Animal {
    constructor(name) {
        this.name = name;
    }
    eat() {
        console.log(this.name + " eats");
    }
}

class Fish extends Animal {
    swim() {
        console.log(this.name + " swims");
    }
}

// (Mixin)
const canJump = {
    jump() {
```

```
        console.log(this.name + " jumps");
    }
};
Object.assign(Fish.prototype, canJump);

const f = new Fish("Nemo");
f.eat();
f.swim();
f.jump();

console.log("\n===Multiple Inheritance===");
const Father = {
    fatherProperty() {
        console.log("Property inherited from Father.");
    }
};

const Mother = {
    motherProperty() {
        console.log("Property inherited from Mother.");
    }
};

class ChildMultiple {
}
Object.assign(ChildMultiple.prototype, Father, Mother);

const childMultiple = new ChildMultiple();

childMultiple.fatherProperty();
childMultiple.motherProperty();
```

7.CODE EXPLANATION

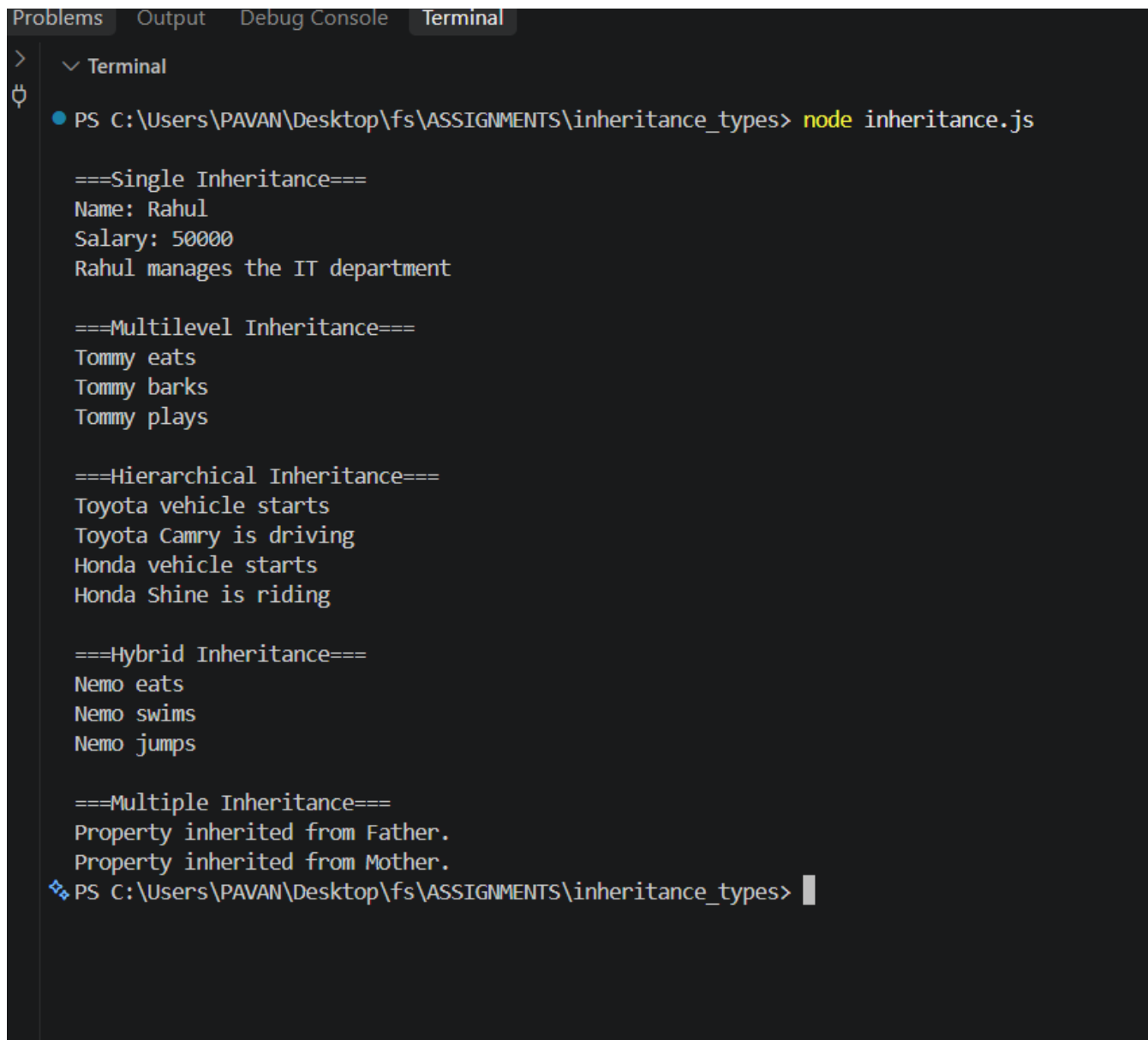
Code	Explanation
Class	Creates a new class, acting as a blueprint for creating objects.
Extends	Establishes a parent-child relationship between classes for classical inheritance.
super()	Excutes the parent class constructor and gives access to its methods.
constructor()	A special function that runs automatically to initialize properties when a new object is created.
this	Points directly to the specific instance of the object currently being executed.
Object.assign()	Merges properties and methods from source objects into a target object, useful for .
method()	A function defined inside a class that defines the behaviors or actions an object can perform.

8.TEST CASES AND EXECUTION DATA

The assignment scripts were executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Create Manager("Rahul", 50000, "IT")	Employee details and department work should be displayed	Displayed correctly	Pass
TC-02	Create Puppy("Tommy", "Beagle", 1)	Puppy should eat, bark, and play	Displayed correctly	Pass
TC-03	Create Car and Bike objects	Vehicle should start and perform respective actions	Displayed correctly	Pass
TC-04	Create Fish object "Nemo"	Fish should eat, swim, and jump	Displayed correctly	Pass
TC-05	Create ChildMultiple object	Father and Mother properties should be displayed	Displayed correctly	Pass

9.OUTPUT



```
Problems Output Debug Console Terminal
>
  Terminal
  PS C:\Users\PAVAN\Desktop\fs\ASSIGNMENTS\inheritance_types> node inheritance.js

===Single Inheritance===
Name: Rahul
Salary: 50000
Rahul manages the IT department

===Multilevel Inheritance===
Tommy eats
Tommy barks
Tommy plays

===Hierarchical Inheritance===
Toyota vehicle starts
Toyota Camry is driving
Honda vehicle starts
Honda Shine is riding

===Hybrid Inheritance===
Nemo eats
Nemo swims
Nemo jumps

===Multiple Inheritance===
Property inherited from Father.
Property inherited from Mother.
PS C:\Users\PAVAN\Desktop\fs\ASSIGNMENTS\inheritance_types> |
```

7.RESULT

Thus, different types of inheritance in JavaScript were successfully implemented and tested. All test cases produced the expected results.